LAB 2 Final Report
Algorithm Performance Measurement and Benchmarking Tool

Student: Harinand S

Roll no: 2401201056

Course: BCA (AI & Data Science)

Lab 2

1. Introduction

Algorithm performance measurement is necessary because the cost of an algorithm can change significantly as input size increases. This project implements a reusable benchmarking tool that measures execution time, peak traced memory and operation counts, and visualizes observed growth.

2. Objectives

- Measure execution time and memory consumption.
- Record comparisons/operations where applicable.
- Compare theoretical and experimental complexity.
- Analyze input-size effects and scalability.
- Generate charts and an interactive Streamlit application.

3. Project Structure

- app.py — Streamlit interface
- search_analysis.py — Linear and Binary Search
- code_benchmark.py — loop and factorial programs
- benchmark_engine.py — automated measurement engine
- project_notebook.ipynb — notebook
- requirements.txt — required libraries
- data/ — benchmark CSV
- graphs/ — performance charts
- screenshots/ — output captures

4. Complexity Comparison

Algorithm	Best	Average	Worst	Space	Growth
Linear Search	O(1)	O(n)	O(n)	O(1)	Linear
Binary Search	O(1)	O(log n)	O(log n)	O(1)	Logarithmic
Single Loop	O(n)	O(n)	O(n)	O(1)	Linear
Nested Loop	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Quadratic
Recursive Factorial	O(n)	O(n)	O(n)	O(n)	Linear + stack
Iterative Factorial	O(n)	O(n)	O(n)	O(1)	Linear

5. Experimental Results

The CSV in this submission was produced by executing the included implementations in the current Python environment. Timings and traced-memory values are machine-dependent observations.

Algorithm	n	Time(s)	Peak KB	Operations	Status
Linear Search	100	0.00000989	0.16	100	executed
Binary Search	100	0.00000272	0.00	6	executed
Single Loop	100	0.00001816	0.10	100	executed
Nested Loop	100	0.00463748	0.19	10000	executed
Linear Search	1000	0.00033125	0.21	1000	executed
Binary Search	1000	0.00000545	0.09	9	executed
Single Loop	1000	0.00037590	0.13	1000	executed
Nested Loop	1000	0.68958625	0.28	1000000	executed
Linear Search	10000	0.00445715	0.21	10000	executed
Binary Search	10000	0.00001072	0.12	13	executed
Single Loop	10000	0.00390374	0.13	10000	executed
Nested Loop	10000			100000000	skipped
Linear Search	50000	0.02154538	0.21	50000	executed
Binary Search	50000	0.00001519	0.12	15	executed
Single Loop	50000	0.02008522	0.14	50000	executed
Nested Loop	50000			2500000000	skipped
Factorial (Recursive)	100	0.00007815	0.20	100	executed
Factorial (Iterative)	100	0.00002857	0.23	99	executed
Factorial (Recursive)	500	0.00112715	8.09	500	executed
Factorial (Iterative)	500	0.00019177	1.11	499	executed
Factorial (Recursive)	900	0.00403396	20.59	900	executed
Factorial (Iterative)	900	0.00078767	2.09	899	executed

6. Search Algorithms

Linear Search checks elements sequentially and has O(n) worst-case time. Binary Search operates on sorted data and halves the search interval, giving O(log n) average/worst-case time for the iterative version. The absent target -1 was used to show worst-case comparisons.

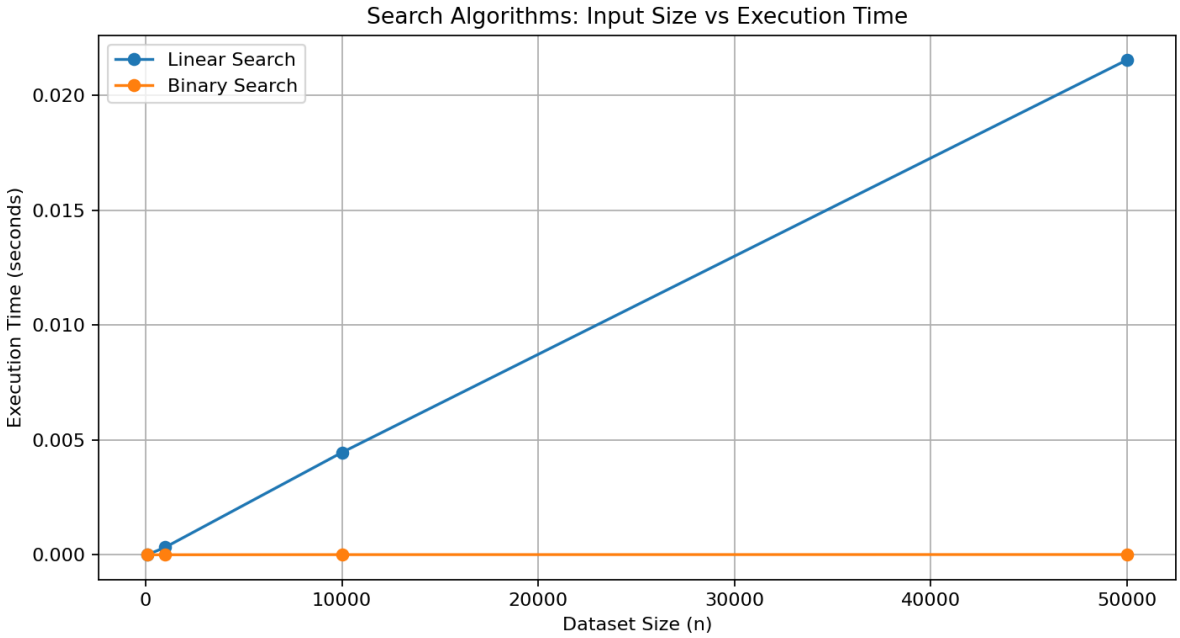


Figure 1. Search execution-time comparison.

Streamlit Application Output Preview

Algorithm Performance Measurement & Benchmarking Tool

Module: Search Algorithms
Dataset size: 50,000
Target: -1 (worst-case / absent)

Linear Search comparisons: 50,000
Binary Search comparisons: 16

Available charts: execution time, memory usage, factorial comparison, loop comparison

Figure 2. Streamlit application output preview.

7. Factorial Programs

Recursive and iterative factorial both have O(n) time. Recursive factorial uses O(n) call-stack space while the iterative implementation uses O(1) auxiliary space.

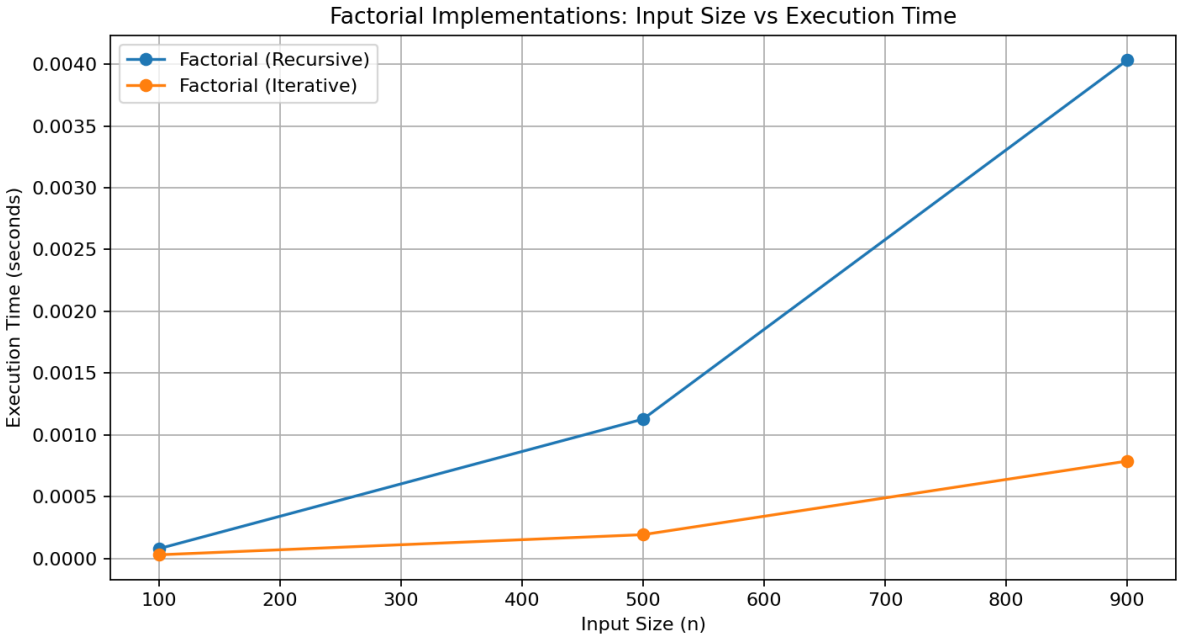


Figure 3. Factorial execution-time comparison.

8. Loop-Based Programs

Single Loop is O(n), while Nested Loop is O(n^2). The quadratic pattern grows much faster. A literal n=50,000 nested loop would require 2.5 billion iterations, so that case is retained in the configuration but intentionally skipped in the executable benchmark. This avoids a misleading or impractical run and is explicitly recorded in the results.

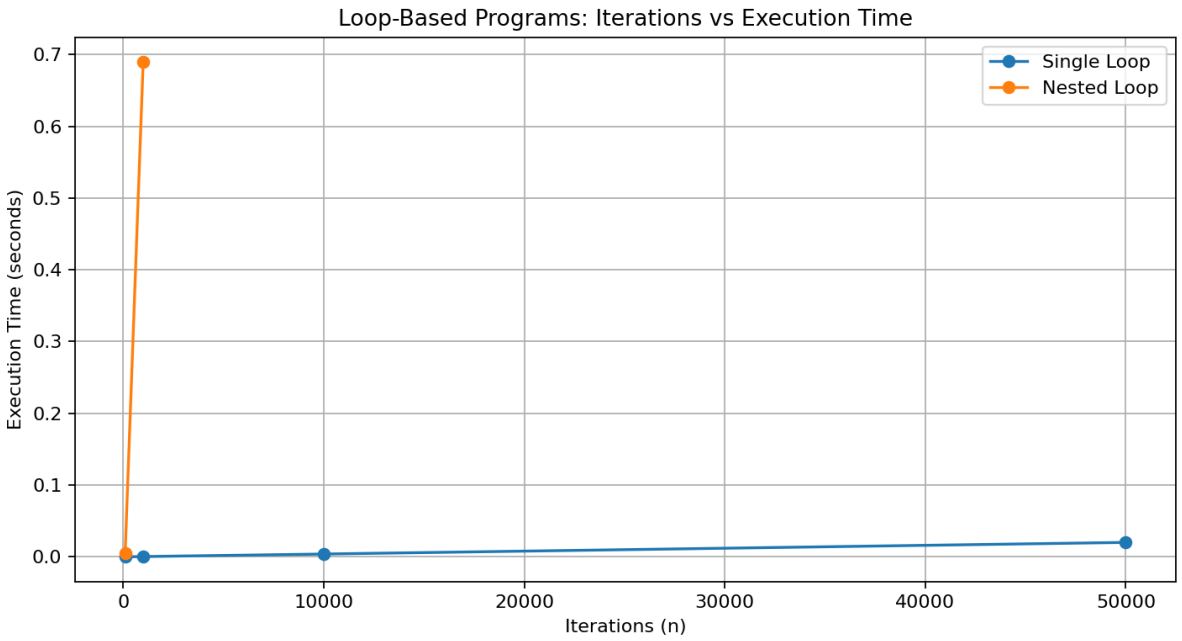


Figure 4. Loop execution-time comparison for executable sizes.

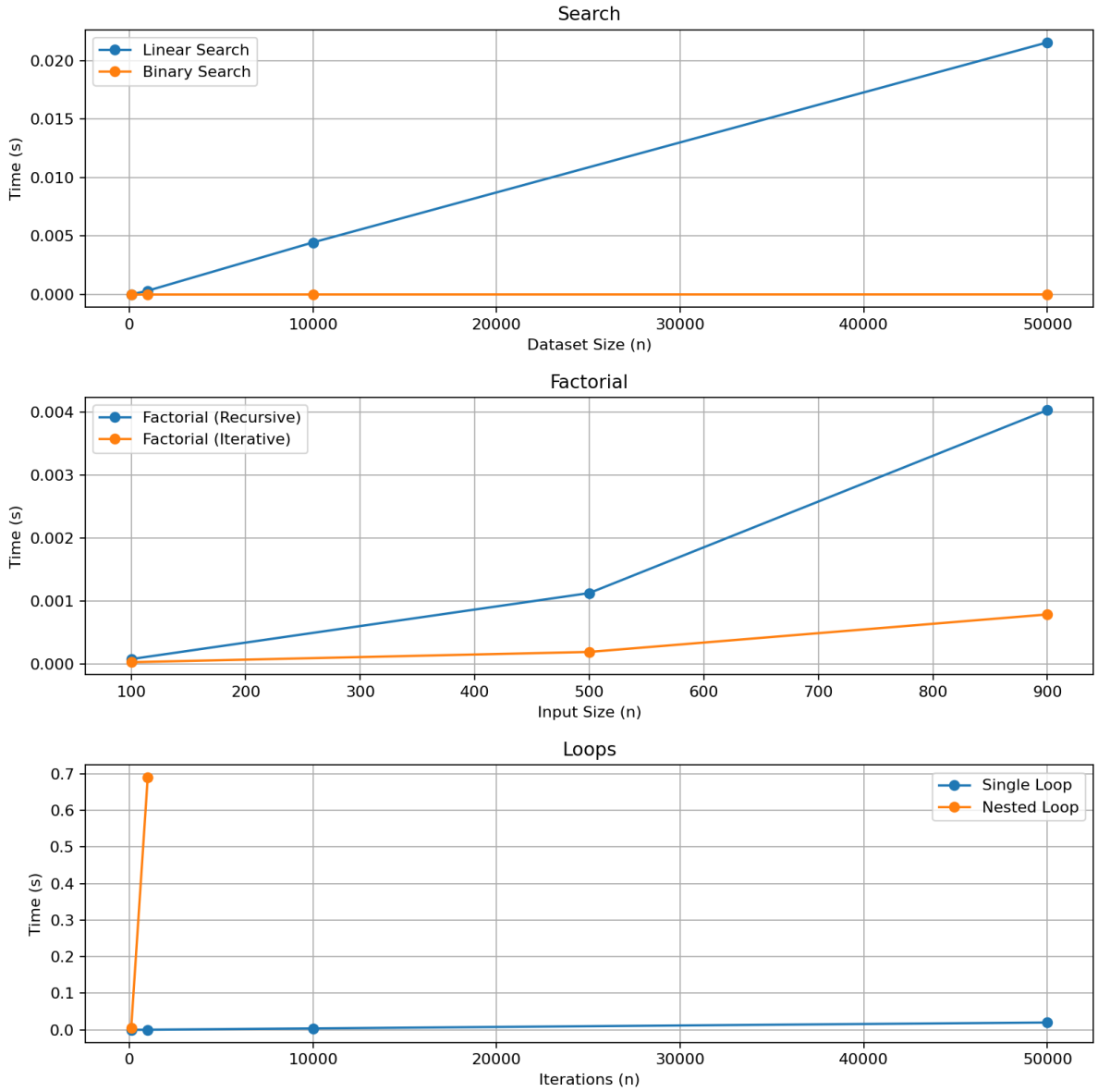


Figure 5. Combined benchmark dashboard.

9. Performance Observations

- Binary Search requires dramatically fewer comparisons for large sorted datasets than Linear Search.
- Single Loop scales linearly, while Nested Loop scales quadratically.
- Recursive Factorial consumes additional call-stack space even though its time complexity is linear.
- Iterative Factorial avoids recursion-stack growth and is preferable when auxiliary-space usage matters.
- Execution time and tracemalloc memory are experimental measurements and vary with the execution environment.
- Asymptotic complexity predicts growth trends rather than a fixed number of seconds.

10. Conclusion

The completed tool demonstrates the relationship between theoretical complexity and practical performance. The search experiment illustrates logarithmic versus linear growth, the loop experiment demonstrates the scalability cost of quadratic work, and the factorial experiment illustrates a time-space trade-off between recursive and iterative implementations.

11. Future Enhancements

- Repeat each benchmark and report mean/median/standard deviation.
- Add sorting, graph and dynamic-programming algorithms.
- Add custom dataset upload and export.
- Record CPU/Python environment information.
- Add interactive statistical and memory visualizations.

12. Evidence

The screenshots directory contains benchmark output, complexity output and application-output captures generated from this submission. Graphs are generated from the included benchmark CSV.